ELLY CHIEF ORCHESTRATOR ROADMAP

ZIELBILD

Elly wird zur zentralen Orchestrator-Schicht von Elyra Core.

Rollenbild
- Elly kennt den Langzeitkontext pro Workspace: Privat, Fish Delish, Elyra Core.
- Elly hlt den groen Plan: Ziele, Prioritten, Initiativen, Routinen, offene Loops, relevante Personen, Hobbys, Arbeitsstil.
- Elly reasoningt r Domains hinweg: Mail, Calendar, Tasks, Projects, Code, Builder, Integrationen.
- Elly zerlegt Absichten in ausfhrbare Task Contracts.
- Spezialisierte Executor fhren diese Contracts aus: Coder, Mail, Calendar, Tasks, Builder.
- Kritische Schritte bleiben supervised: Send, Push, Merge, Promotion, deletions, money-moving actions.

Zielzustand im Betrieb
1. Du sagst Elly: Check meine Mails, fasse zusammen, bereite Antworten vor, plane meine Woche.
2. Elly liest Workspace Brain + aktuelle Mail/Calendar/Tasks/Project-Kontexte.
3. Elly erzeugt einen priorisierten Plan mit konkreten Subtasks.
4. Elly delegiert kleine, klar gescopte Auftrge an den Coder oder andere Apps.
5. Die Executor liefern Events, Artefakte, Diffs, Drafts und Verifikationsresultate zurcck.
6. Elly aktualisiert das Langzeitgedchtnis und empfiehlt den nchsten Schritt.

VORHANDENE MODULE WIEDERVERWENDEN

1. Workspace Brain und Identitt
Bereits vorhanden
- frontend/app/api/v1/workspace-brain/route.ts
- workspace_identity.profile_json
- Settings/Brain-Surface in components/os/shell/SettingsView.tsx

Wiederverwendung
- workspace_identity.profile_json wird zum SSOT fr Workspace-Strategie.
- Das bestehende Brain-API bleibt der Einstiegspunkt fr Lesen/Schreiben.
- Die Settings-Oberflche wird zur Editor-Flche fr Workspace-Ziele, Operating Rules und Rollenbild.

2. Runs, Events, Approvals, Audit
Bereits vorhanden
- frontend/app/api/os/runs/quick-action/route.ts
- orchestration_runs
- run_events
- Worker-Runtime in worker/src/cloudOs.ts

Wiederverwendung
- orchestration_runs bleibt die zentrale Run-Ledger-Tabelle.
- run_events bleibt die Event-Timeline fr UI, Audit und Debugging.
- Quick actions werden nicht ersetzt, sondern unter den Chief-Orchestrator gehngt.

3. Elli Intent- und Action-System
Bereits vorhanden
- worker/src/elli/system.ts

Wiederverwendung
- Intent-Klassifikation bleibt erhalten.
- assistant, planner, operator werden nicht neu erfunden.

- Das System wird um cross-domain reasoning, workspace memory retrieval und cont
ract generation erweitert.

4. Mail, Calendar, Tasks
Bereits vorhanden
- Worker-Endpoints in worker/src/index.ts
- Frontend-Apps in MailProgram.tsx, CalendarProgram.tsx, TasksApp.tsx

Wiederverwendung
- Mail, Calendar und Tasks werden als erste-class Executors behandelt.
- Elly ruft keine diffuse Generallogik auf, sondern delegiert an bestehende Domn
en-APIs.
- Draft- und Approval-Flows bleiben supervised.

5. Coder und lokaler Agent
Bereits vorhanden
- frontend/app/api/os/code/run/route.ts
- worker/src/apiServer.ts
- worker/src/devTaskRunner.ts
- worker/src/features/featureExecutor.ts

Wiederverwendung
- Der Coder bleibt Executor fr Codearbeit.
- Die neue Aufgabe des Chief-Orchestrators ist nur, klare Task Contracts an den
Coder zu bergeben.
- Scoping, file edits, terminal, build, test, git und push bleiben im Coder.

6. Provider-Routing
Bereits vorhanden
- frontend/lib/orb-os/providers/providerRouter.ts

Wiederverwendung
- Provider-Routing bleibt Plattformfunktion von Elyra Core.
- Gemini, Codex, Claude, spter Mistral/Perplexity laufen unter demselben Run-Rah
men.
- Unterschiede werden als Capabilities modelliert, nicht als komplett eigene Abl
fe.

7. Builder / Workflow-Flche
Bereits vorhanden
- components/os/apps/builder/BuilderWindow.tsx
- components/os/apps/studio/StudioWindow.tsx

Wiederverwendung
- Builder wird nicht als Demo belassen, sondern mit echten automations und tool
actions verdrahtet.
- Studio kann Skill-/Workflow-Authoring fr wiederkehrende Ablfe werden.

NEUE SERVICES, TABELLEN UND APIS

A. Chief Orchestrator Service
- worker/src/orchestrator/chiefOrchestrator.ts
Aufgaben
- Workspace Brain laden
- aktuelle Kontext-Snapshots laden
- Intent in operative Mission umwandeln
- Task Contracts erzeugen
- Executor auswhlen
- Supervisor-Entscheidungen markieren
- Memory-Updates nach erfolgreicher Ausfhrung schreiben

B. Workspace Brain v2

Neue Struktur im Brain-Profil
- mission
- goals
- activeInitiatives
- operatingRules
- peopleContext
- routines
- hobbies
- preferences
- openLoops
- delegationPolicy

Neue API
- GET /api/v1/workspace-brain/context?workspaceId=...
- POST /api/v1/workspace-brain/upsert-memory
- POST /api/v1/workspace-brain/derive-summary

C. Context Snapshot Service
Neue Tabelle
- workspace_context_snapshots

Zweck
- voraggregierte Snapshots fr unread mail summary, next calendar blocks, urgent tasks, active projects, recent coding outcomes

Neue Jobs/APIs
- POST /api/os/context/refresh
- GET /api/os/context/snapshot?workspaceId=...

D. Task Contract Schema
Neue Tabelle
- task_contracts

Felder
- id
- workspace_id
- run_id
- executor_type
- title
- goal
- scope_json
- constraints_json
- definition_of_done_json
- verification_json
- priority
- status

Neue API
- POST /api/os/contracts
- GET /api/os/contracts/[id]
- POST /api/os/contracts/[id]/dispatch

E. Executor Registry
- worker/src/orchestrator/executorRegistry.ts
Mappt Contract-Typ auf Executor: code, mail, calendar, tasks, workflow

F. Provider Capability Registry
- frontend/lib/orb-os/providers/capabilityRegistry.ts
Capabilities
- speed
- tool-use quality
- long-context fit

- code-edit reliability
- JSON reliability
- review quality
- cost band

G. Memory Writeback Ledger
Neue Tabelle
- workspace_memory_entries
Typen
- decision
- preference
- routine
- project_state
- people_note
- coding_decision
- follow_up

H. Workflow Runtime
Neue Tabellen
- workflow_definitions
- workflow_runs
- workflow_steps
- workflow_step_events

Neue APIs
- POST /api/os/workflows
- POST /api/os/workflows/[id]/run
- GET /api/os/workflows/[id]/runs

REIHENFOLGE DER UMSETZUNG

Phase 1: Coder Reliability und Truth Fixes
Dauer: 1 bis 2 Wochen
Ziele
- Preview, build, tests und applied changes haben eine einzige Wahrheit.
- Scoped runs nutzen standardmig den schnellen Tool-first Pfad.
- Groe Planung wird nicht im Coder erzwungen.
Arbeitspakete
- Coder Fast Path fr kleine/scoped Tasks
- Vollstndige Spiegelung des verifizierten Mirrors in Preview
- Harte Scope-Erzwingung bei UI-Fixes
- Live Run Events stabilisieren
- echte Build-/Webpack-Failure-Propagation in UI

Phase 2: Workspace Brain als SSOT
Dauer: 1 bis 2 Wochen
Ziele
- einheitliche Brain-Struktur fr Privat, Fish Delish, Elyra Core
- der Orchestrator kennt Mission, Ziele, Routinen und Prioritten
Arbeitspakete
- Brain v2 Schema definieren
- vorhandene workspace_identity.profile_json Migration
- Brain Read-Model API einfhren
- Settings-Oberflche zum Editieren des Brain ausbauen
- Memory writeback ledger einfhren

Phase 3: Chief Orchestrator
Dauer: 2 bis 3 Wochen
Ziele
- Elly denkt den groen Plan, nicht der Coder
- Elly erzeugt Contracts fr spezialisierte Executor
Arbeitspakete

- chiefOrchestrator.ts einfhren
- Context snapshot loading anbinden
- Task contract schema + dispatch einbauen
- Run-Typ chief_orchestration einfhren
- approval-/supervision-Layer definieren

Phase 4: Domain Executors zusammenziehen
Dauer: 2 bis 4 Wochen
Ziele
- Mail, Calendar, Tasks und Coder arbeiten unter einem gemeinsamen Executor-Mode
ll
Arbeitspakete
- Executor registry bauen
- Mail draft/send supervised flows standardisieren
- Calendar plan/create/reschedule standardisieren
- Tasks derive/create/update standardisieren
- Ergebnisformate vereinheitlichen

Phase 5: Workflow Runtime und Builder
Dauer: 2 bis 3 Wochen
Ziele
- Elly kann mit dir wiederkehrende Workflows definieren und ausfhren
Arbeitspakete
- Workflow runtime Tabellen und APIs
- Builder an echte workflow definitions anbinden
- Chief Orchestrator kann Workflow-Vorschlge erzeugen
- supervised approvals in Workflow-Schritten

Phase 6: Platform Hardening
Dauer: 3 bis 6 Wochen
Ziele
- hohe Zuverlssigkeit im Alltag
Arbeitspakete
- Retry-Strategien
- Idempotenz fr actions
- bessere Fehlerklassifikation
- quotas, rate-limit, backoff
- monitoring und incident diagnostics
- provider capability routing finalisieren

PERFORMANCE-STRATEGIE PRO PHASE

Phase 1
- keine Pflicht-Planung fr kleine Tasks
- Full-repo scan vermeiden, wenn Scope bekannt ist
- direct-edit/tool-first vor feature-agent fallback
- Preview- und Verification-Pfade entdoppeln
Messwerte
- time-to-first-edit
- total run time for scoped UI fixes
- failed runs due to patch/apply mismatch

Phase 2
- Brain Read-Model cachen
- Workspace-Profil klein und strukturiert halten
- schwere Ableitungen in Background-Summaries auslagern
Messwerte
- brain load latency
- cache hit rate
- number of full profile rehydrates per run

Phase 3

- Kontext-Snapshots statt Live-Vollabfragen
- Orchestrator erzeugt nur Contracts, nicht die ganze Execution selbst
- Contracts klein und streng typisiert halten
Messwerte
- time-to-plan
- contracts generated per run
- average contract size

Phase 4
- einheitliche executor interface
- domain-specific logic bleibt in den jeweiligen Modulen
- nur das Handoff standardisieren
Messwerte
- dispatch latency
- executor success rate
- approval turnaround time

Phase 5
- asynchron ausfhren
- step events append-only
- nur relevante Schritte streamen
- snapshots statt Vollhistorien im UI
Messwerte
- workflow run latency
- step throughput
- event stream volume per run

Phase 6
- provider routing nach capability und cost band
- retries nur dort, wo sie Erfolgschance haben
- evidence-first debugging
- budget und quota enforcement frh im Run
Messwerte
- p95 run duration
- cost per successful task
- retry success rate
- incident count by executor

WICHTIGSTE ARCHITEKTURENTSCHEIDUNG
- Elly ist der Chef-Orchestrator.
- Coder ist ein Executor.
- Mail/Calendar/Tasks/Builder sind ebenfalls Executor.
- Workspace Brain ist das gemeinsame Langzeitgedchtnis.
- Runs, events, approvals und audit bleiben das Plattform-Rcckgrat.

Wenn dieser Schnitt sauber umgesetzt wird, steigen Zuverlssigkeit und Geschwindigkeit.